第四部分 · 记忆的表示与存储架构

向量记忆与向量数据库

如何从海量历史中快速寻找相关记忆?

Vector Memory · ANN · HNSW · IVF · Product Quantization

> 记忆的候选查找可以很快，
> 但“找得相似”不等于“记得正确”。

零基础理论课 · 18张原理图 · 12道思考题及解析

v1.0 | 2026-10-18 | 纯理论，无实验

第18讲｜向量记忆与向量数据库

《智能体记忆系统：从零基础到科研理解》 · 第18/32讲 · 第四部分：记忆的表示、存储架构与检索机制

副标题：如何从海量历史中快速寻找相关记忆?

本讲的核心问题： 如果智能体有10万条甚至100万条长期记忆，为什么不能每次都拿出来读？计算机怎样在不逐条完整比较的情况下快速找到“可能相关”的记录？加速检索会付出什么代价？检索到的记忆又为什么不一定可信、有效或适合当前任务?

阅读约定：本讲不安排实验和编程。所有D05、D18、D30之类的项目时间线、计算量与向量坐标均为**教学虚构**；没有将其当作真实模型测试。HNSW、IVF、PQ等算法的机制依据来自原始论文与官方文档，末尾提供完整可访问来源。

学习目标与本讲路线

学完后应当能够：①独立画出向量记忆系统的写入和检索路径；②解释精确最近邻与近似最近邻的区别；③理解HNSW、IVF与PQ分别解决哪一层问题；④推导向量容量与Recall@K；⑤说明元数据过滤、版本更新和索引延迟怎样影响可靠性；⑥比较“索引检索质量”与“智能体行动质量”；⑦知道什么情况下反而不需要昂贵的向量数据库。

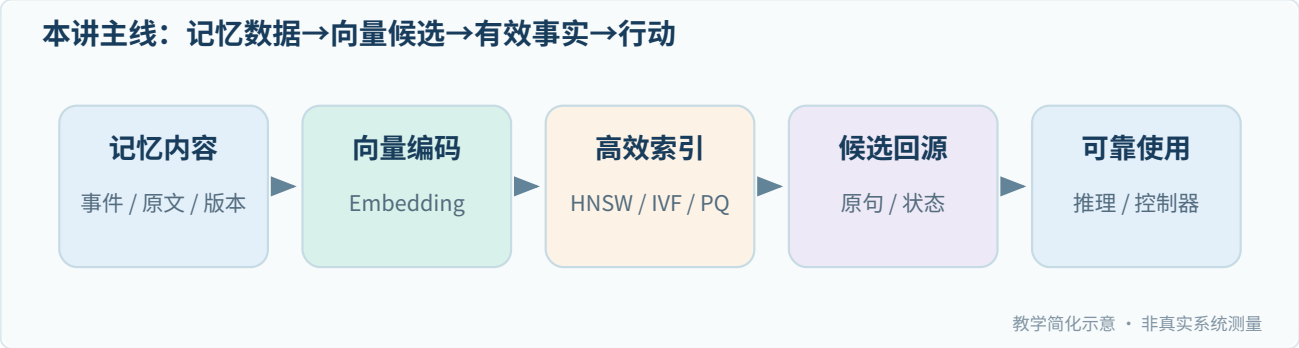


图1 第18讲完整知识路线｜教学机制示意，非实际模型运行截图

> **总原则：**向量索引帮助系统以更少的计算找到“候选记忆”；**它本身不负责判定历史信息是否真实、最新、获得授权或足以支持行动**。真正可靠的记忆系统还需要原文、元数据、版本、权限以及推理后的控制器核验。

一、先遇到真实规模的问题：记忆多了以后会怎样?

假设一个科研智能体持续辅助某研究项目，长期积累了**100万条候选记忆**。这些记忆里有论文片段、聊天事件、工具结果、项目决定、错误排查和摘要。以下是教学虚构时间线：

- **第05天**：智能体建议“之后可以考虑购买API额度、尝试扩样”，这只是AI提议。
- **第18天**：研究者正式决定“暂停一切未经重新授权的付费调用”，这是一个当前生效的限制。

- **第24天**：工具日志发现有一次HTTP 504错误，原因仍需要核查。
- **第30天**：研究者问“结合之前的讨论，现在能不能继续发起付费实验？”

系统应同时找到第05天的历史提议、第18天的有效决定以及最近的其他证据，再据此作答。但如果逐一把100万条记录复制到提示词，不仅可能远超上下文窗口，还会消耗大量Token，并把大量无关信息混入模型。要解决的是**从海量历史中选出少量相关候选**，不是把信息全部消灭。

假设每条记录编码为768维浮点向量：100万条仅向量数据的量级已经达到数GB，正文、索引边和元数据另计。一次提问也不会为了回答一句话而加载全部原始文本。这里就需要针对向量的高效检索结构。

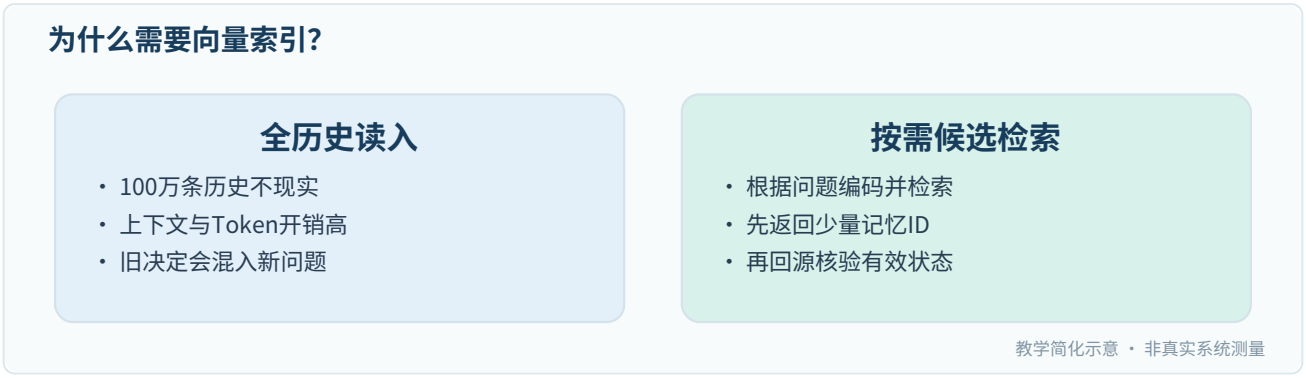


图2 全量历史回放与按需检索｜教学机制示意，非实际模型运行截图

1.1 为什么普通的“文件夹搜索”还不够?

传统关键词检索擅长“HTTP 504”“实验编号E17”这样的精确词项；向量检索尝试让“能继续跑收费实验吗”与“暂停未经许可的付费API请求”发生语义匹配，即使文字不同。两者各有盲区：向量相似不保证逻辑相容，关键词也可能漏掉同义改写。因此本讲研究的是向量检索的内部技术，不意味着全系统应该放弃关键词或结构化条件。

1.2 三层职责先分开

记忆内容层保存可回溯的文本/事件、来源与版本；**向量检索层**依据向量距离快速筛出候选ID；**记忆使用层**检查当前状态、权限、来源与证据，再交给模型推理与控制器执行。后面所有HNSW、IVF、PQ主要处于第二层。

二、从文本到向量记忆：究竟存了什么?

第07讲已介绍Embedding。现在给它一个工程角色：把可能相关的文本或结构化记忆描述编码成固定维度向量，并将向量与稳定的`memory_id`建立关联。

例如一条记忆卡有：`memory_id=M018`；原文“第18天暂停新增付费调用”；`project=P`；`status=ACTIVE`；`speaker=research_lead`；`valid_from=D18`；`source_event=E018`；`embedding=[...]`。真实向量通常有数百或上千个数，图中只用2个数表达几何直觉。

向量是索引中的可比较表示，不是事件本身。只保留向量会丢失可引用的原句、时间限制、具体数值、权威来源等信息。数据库可以把向量与正文放在一张表，也可以采用“权威记录库+向量索引服务”；这只是物理部署选择。

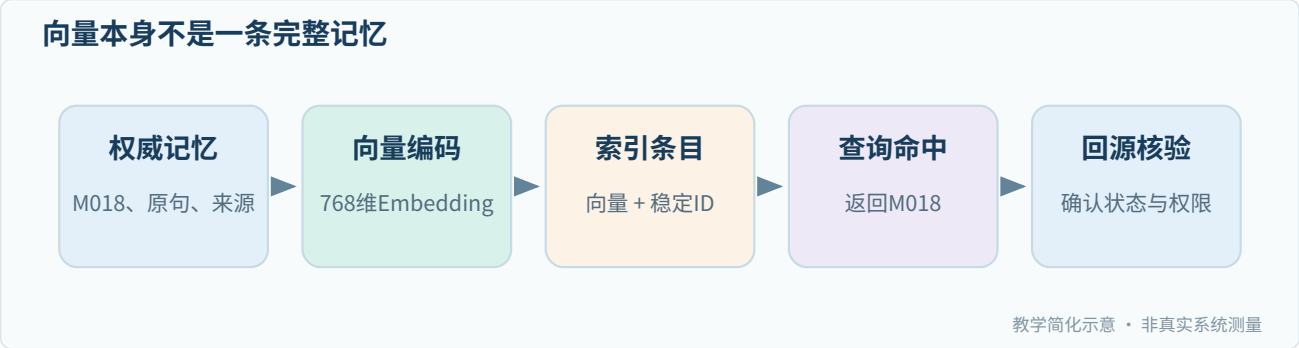


图3 记忆条目与向量索引的关联｜教学机制示意，非实际模型运行截图

2.1 查询也是一条向量

当用户说“付费实验还能开展吗？”，检索器用与索引兼容的Embedding模型编码查询为向量q，再计算q与记忆向量v_i之间的距离/相似度，返回一批候选ID。**编码模型和配置必须兼容**：不同模型的维度、归一化与语义空间通常不能任意混用；直接把旧索引接给新版Embedding模型可能造成错误排序。

2.2 距离、相似度与排序方向

三种常用指标：

- **欧氏距离（L2）**：$d(q,v)=\sqrt{\Sigma(q_i-v_i)^2}$，距离越小越近。
- **点积**：$s(q,v)=\Sigma q_i v_i$，一般越大越相似，但受向量长度影响。
- **余弦相似度**：$\cos(q,v)=q\cdot v/(||q||\times||v||)$，衡量夹角，越大越接近同一方向；零向量需要特殊处理。

若q=(1,0)，A=(2,0)，B=(0.8,0.6)，那么余弦(q,A)=1、余弦(q,B)=0.8，而点积分别为2和0.8。A在两个分数中都领先，但这不证明A更真实；若比较C=(0.9,0)，C与q余弦也为1，而欧氏距离只有0.1。**比较排名前必须明确距离公式及向量是否归一化**。

对于所有向量都经L2归一化为单位长度的情况，点积与余弦相同，欧氏距离平方$||q-v||^2=2-2(q\cdot v)$，因此三者的排序关系可等价；没有归一化就不能随意声称等价。

三、最朴素也最可靠的基线：精确最近邻

最近邻（Nearest Neighbor, NN）就是给定q，找到距离最近的一条向量；**Top-K/K近邻（k-NN）**是返回最近的K条。精确搜索（Exact Search）需要根据定义比较所有符合条件的向量，常称**暴力扫描/Flat Search**。

用5条二维教学向量可以手算：查询q=(1,0)；A=(1,0)、B=(0.8,0.2)、C=(0,1)、D=(-1,0)、E=(0.7,0)。欧氏距离依次为0、约0.283、约1.414、2、0.3，因此精确Top-3顺序为A、B、E。

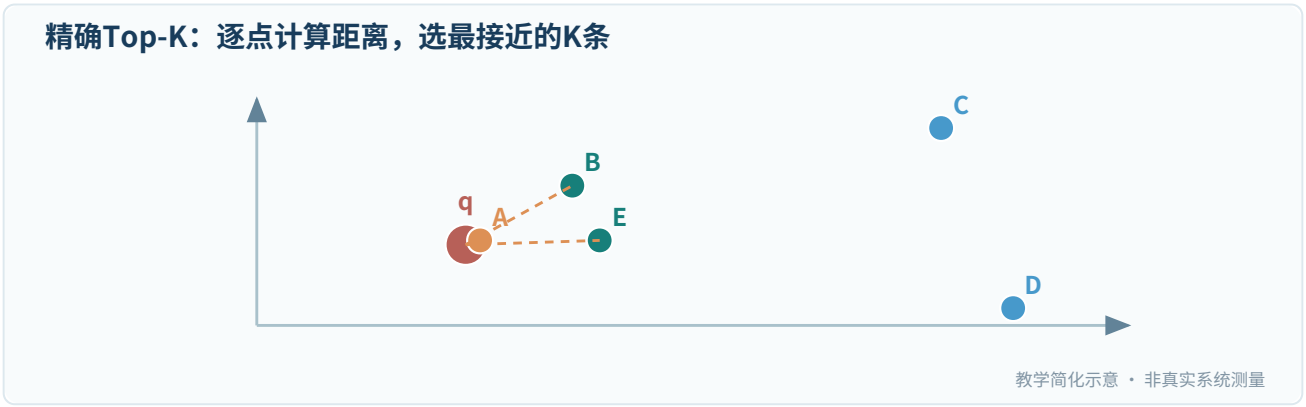


图4 精确近邻示意｜教学机制示意，非实际模型运行截图

3.1 为什么它并不“落后”？

精确扫描没有索引带来的近似遗漏，便于作为评测真值；在几千至几万条记录、查询不频繁、或高速向量矩阵运算可承受的场景，简单扫描可能已足够。构建索引同样有时间、内存与维护成本，并不是“只要有向量就必须上大型向量数据库”。Faiss官方文档明确把Flat索引视为重要基线。[1][2]

3.2 O(Nd)的直观含义

若有N条向量，每条d维，则一次全量计算所需比较工作量随`N×d`增长，写作`O(Nd)`。**大O是增长趋势，不是毫秒预测**：CPU/GPU吞吐、数据布局、内存带宽、批量查询和缓存都会影响实际延迟。

课堂算例：`N=1,000,000`，`d=768`，单精度float32每个数4字节。原始向量空间为：

`N × d × 4 = 1,000,000 × 768 × 4 = 3,072,000,000 字节`。

约为**3.072 GB（十进制）/2.86 GiB（二进制）**，这**不包括**向量ID、图边、元数据、原文、缓存、复制及数据库管理开销。百万条×768维的点积涉及约7.68亿次乘法和类似规模的加法；但“计算次数多”不等于某种硬件一定很慢。

3.3 精确Top-K是ANN评价的一把尺

假设精确搜索得到`[A,B,C,D,E]`，新索引只返回`[A,C,D,F,G]`。双方Top-5交集为`{A,C,D}`，因此**近邻召回率**`Recall@5=3/5=60%`。这表示它找回了精确前5中的3条，并**不是**“问题回答正确率60%”，也不是“模型记忆只有60%”。两种评价目标必须分开。

四、ANN：用可能漏掉一些近邻，换取更快的检索

Approximate Nearest Neighbor（ANN，近似最近邻）不是“随便找几个看上去接近的点”，而是让搜索利用索引结构只访问候选集合的一部分，力图减少比较次数，同时保持可接受的近邻召回。

算法的共同思想可以记成一句话：**先用结构缩小搜索范围，再在少量候选中做更细比较。**典型家族包括图索引、聚类倒排索引、量化索引和混合方法。不同实现不一定遵循相同的访问顺序，也未必具备相同的数据更新功能。

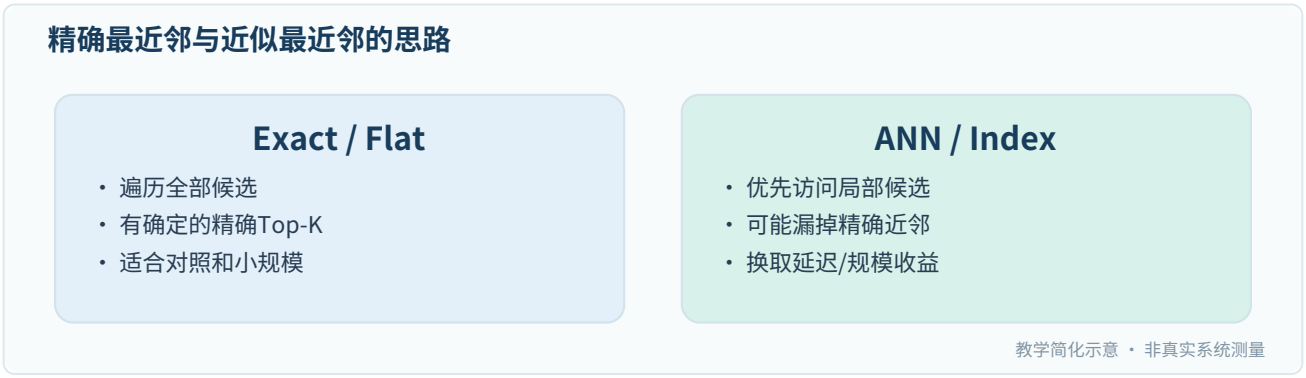


图5 Exact与ANN的基本取舍｜教学机制示意，非实际模型运行截图

4.1 速度—召回率—内存三角关系

减少访问通常更快，但可能漏掉真正最近的点；增加搜索范围往往提高Recall@K，却牺牲延迟和计算；保存更多邻居边/精度信息会占更多内存。索引构建速度、增量写入与删除又增加维度。现实中的最优选择是一条**质量—成本权衡曲线**，不是一种算法永远最好。

4.2 为什么ANN的失败会传到语言模型?

如果最重要的记忆M018根本未进入候选集合，后面的重排器无法凭空补回它；语言模型最终只能在输入中已有的证据上推理，或者产生无依据猜测。反过来，即使ANN完整召回了M018，如果系统没检查时间与权限状态，仍可能错误使用第05天的旧提议。所以要把**候选检索失败**和**记忆状态判定失败**分开。

五、HNSW：让向量结成“能导航的多层关系网”

HNSW全称**Hierarchical Navigable Small World**，可译为“分层可导航小世界图”。由Malkov与Yashunin提出，核心思想是：每个向量作为一个点，与若干相近点建立图边；搜索时从入口点开始沿邻居边逐步靠近查询，借助多层稀疏到稠密的图加速导航。[3]

5.1 先理解没有层级的近邻图

想象一座没有路牌的城市，每栋房子是一条向量，距离相近的房子通过道路连起来。你要到达位置q，却不能查看城市所有房屋的直线距离。可以从入口A出发，比较A的几个邻居，朝更靠近q的方向移动，继续访问。这就是图导航的直觉。

不过，只沿着“眼前更近”的点贪心移动，可能遇到**局部最优**：某处周围邻居都稍远，但跨过它以后存在真正更近的区域。因此实际HNSW搜索会维持候选集合并扩展多条有希望的路径，而不只是课本里一根单线箭头。

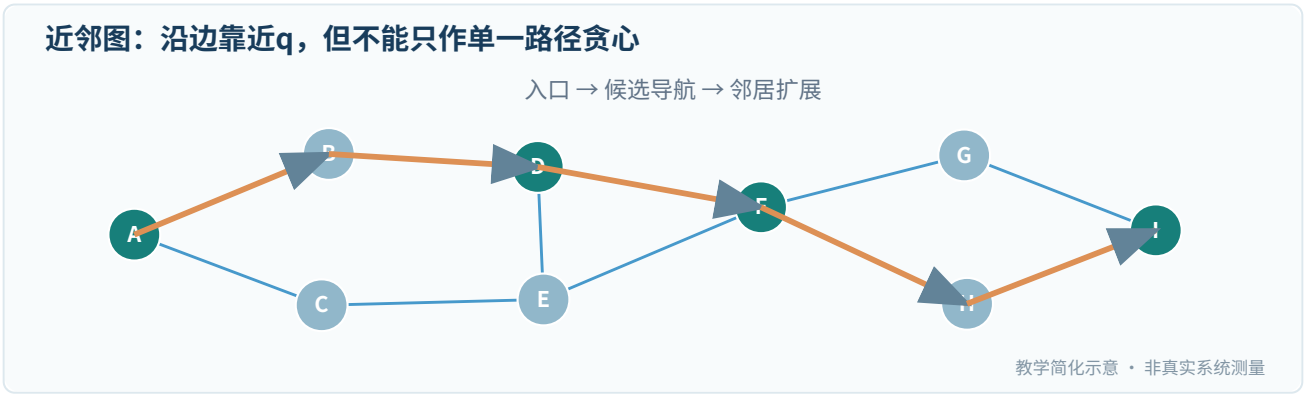


图6 近邻图搜索的导航直觉｜教学机制示意，非实际模型运行截图

5.2 为什么增加多层?

底层图连接大量点，便于精细搜索；高层只包含部分点，连接更稀疏但可以用较少跳数跨越较大区域。查询先在较高层粗定位，再逐层下降到低层细搜。

把它想成交通系统：高速公路负责远距离移动，城市主路负责进入街区，小路负责找到门牌。**这是结构类比，不是HNSW一定与地理网格相同**；具体图边由向量相似性与构图算法决定。

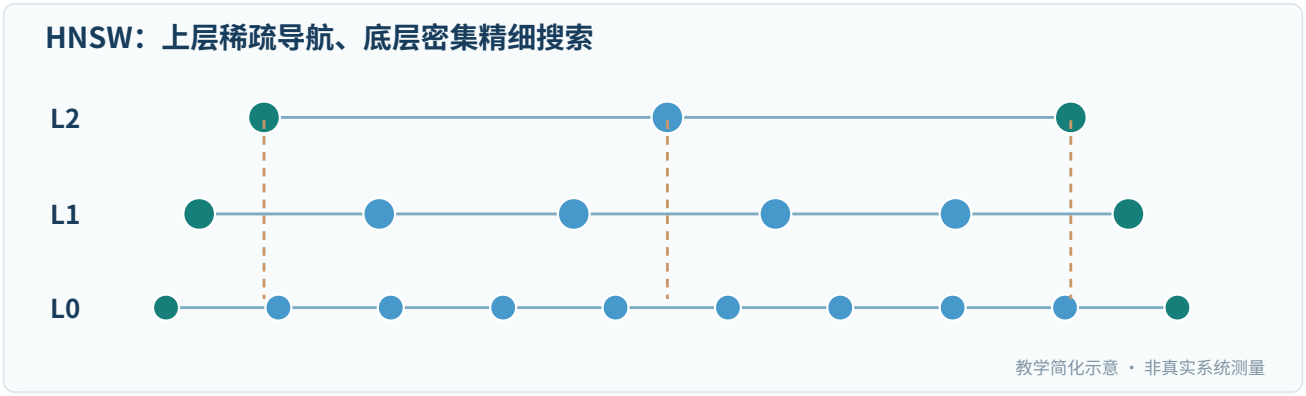


图7 HNSW多层图概念｜教学机制示意，非实际模型运行截图

5.3 构建与搜索是两段不同的过程

构建索引：新增向量，按特定随机层级分布确定其出现的最高层；在各层寻找候选邻居、选择并连接边，必要时更新旧节点的邻接关系。**查询索引**：从入口点自上向下导航，底层进行更广泛的候选搜索，最后返回Top-K。

需要区分三类常见参数：

- `M`：图中节点保持的邻居连接规模相关参数，影响索引内存与可导航性；具体各层上限取决于实现。
- `efConstruction`：构建时的候选搜索范围，通常更大意味着构建更慢、占用资源更多，有机会得到更好的图。
- `efSearch`（pgvector中为`hnsw.ef_search`）：查询阶段候选搜索范围，通常加大可提升召回率、增加查询开销。[3][4]

这些参数不是语言模型的温度或注意力参数；调大也不会自动解决“摘要写反了否定条件”的数据内容错误。

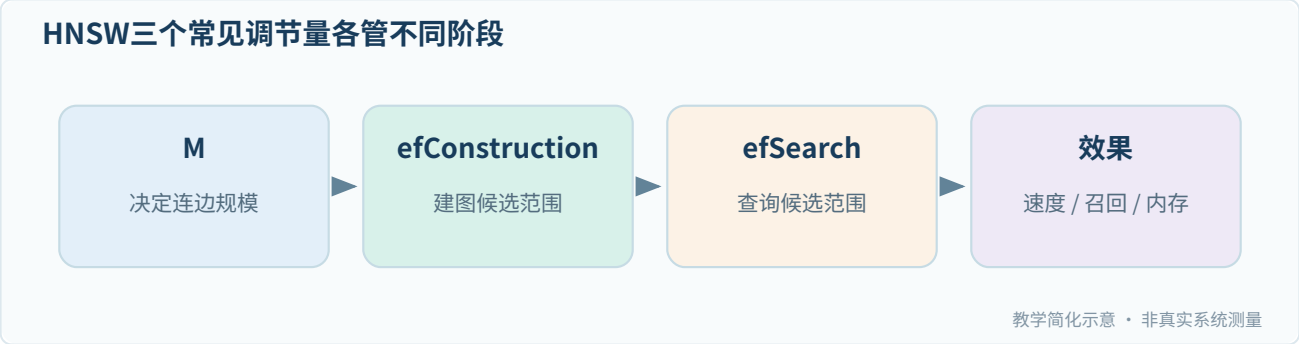


图8 HNSW主要参数作用｜教学机制示意，非实际模型运行截图

5.4 什么时候HNSW未必合适?

图会占内存，构建和插入也有成本；频繁删除和大量更新可能造成维护负担。特别小的数据集直接扫描可能更省事。对低内存、磁盘驻留数据或特殊硬过滤需求，也可以考虑其他索引或精确查询。HNSW常见并不代表它是所有问题的默认最佳解。

六、IVF：把向量先分组，再查可能相关的组

IVF（Inverted File Index，倒排分区索引）这里不是关键词倒排表，而是**向量聚类后建立的“中心点→成员列表”结构**。常见IVFFlat做法：先根据训练样本找到若干中心（常用k-means思路），把每个向量分配给相应的列表；查询先找最接近q的中心，只搜索部分列表。[1][4]

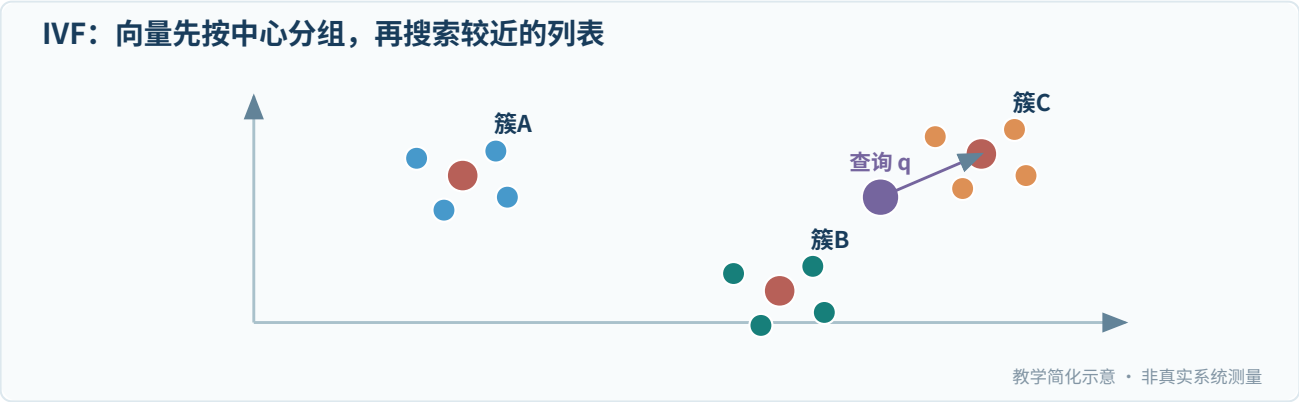


图9 IVF聚类分区检索｜教学机制示意，非实际模型运行截图

6.1 一个12个点、3个区域的二维例子

设有12条记忆向量，被分到“模型与训练”“工具与网络”“项目与权限”三个近似聚类。注意：**真实聚类按数学距离生成，并不知道这些人类语义标签**。查询q靠近“项目与权限”的中心，如果只搜索该列表，能节省很多比较；但落在两组边界的真实近邻可能被漏掉。

`nlist/lists`指列表数量，`nprobe/probes`指查询时探查多少个列表。增大nprobe，通常搜索更全面、召回提高，但延迟也会上升。设3组、每组4条：只探1组最多直接比较本例4条，探3组比较全部12条；真实系统分布可能不均匀，因此“探1/3的中心”不严格等于只访问1/3的数据。

6.2 分区不等于严格语义分类

如果一条“禁止未经授权的收费调用”向量与“过去计划申请API经费”很接近，二者可能被分到同一组；但“禁止”和“申请”的逻辑状态仍要由结构化字段与原文核验。向量聚类提供**搜索候选组织**，不是项目规则的知识分类法。

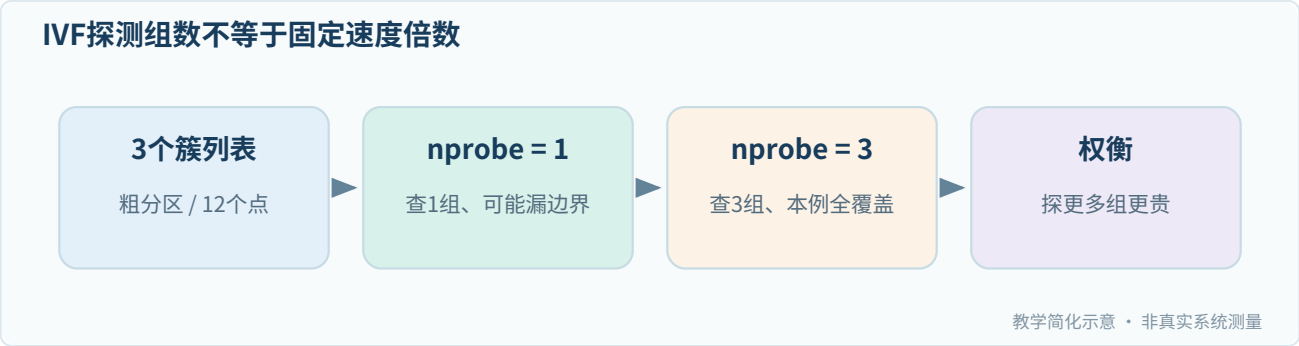


图10 IVF探测列表的覆盖范围｜教学机制示意，非实际模型运行截图

6.3 与HNSW如何比较?

HNSW以图导航为核心；IVF以粗聚类分区为核心。它们可以和量化组合，并非只能二选一。从pgvector官方文档的实现角度，HNSW通常具备较好的速度—召回权衡，但建图更慢、占更多内存；IVFFlat建得快、内存少但效果受训练与列表划分影响。这是具体实现的技术取舍，而不是适用于所有数据集的绝对定理。[4]

七、量化与PQ：给每个向量“压缩编码”

即使ANN避免扫描全部向量，100万条768维float32的原始向量仍占约3.072 GB（十进制）。量化（Quantization）的目标之一，是减少向量表示的存储与内存带宽成本。常见方法有float16、标量量化、二值量化和**Product Quantization（PQ，乘积量化）**。

7.1 先理解“有损”

把浮点数`0.81273`只保留到有限精度，例如`0.81`，占用可能变少，但信息不完全一样。距离计算使用近似数字，部分向量的先后顺序可能变化。因此向量压缩不是“免费缩小硬盘”。

7.2 Product Quantization（PQ）的核心思路

PQ把高维向量划成多个子空间，在每个子空间里使用事先训练好的小型“代表向量表（码本）”，每个子空间只保存最接近哪个码字的**编号**。检索时可以用这些编码近似计算距离。这不是文本摘要，也不会自动保留原始主张。[5]

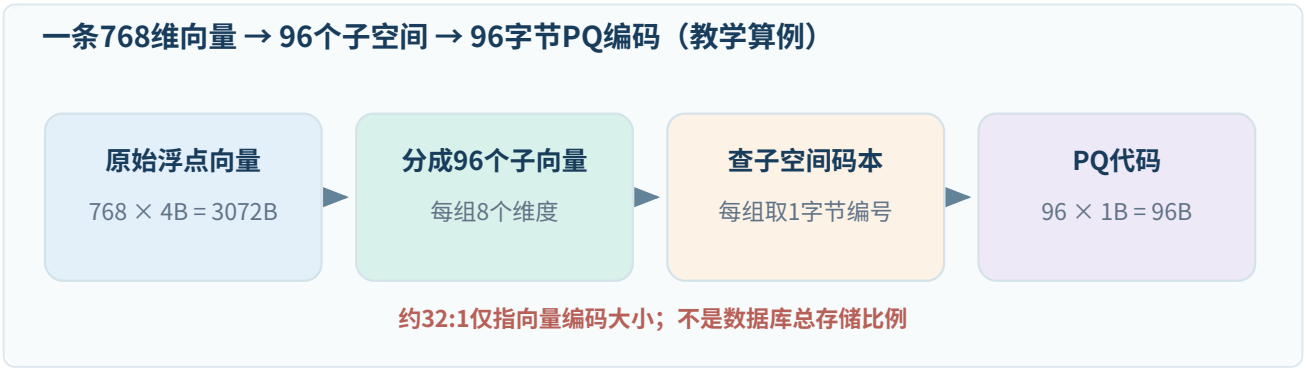


图11 PQ编码直觉与字节预算｜教学机制示意，非实际模型运行截图

课堂空间计算：768维float32一条向量占`768×4=3072字节`。若把它拆成96个各8维的子向量，且每组用**8-bit（1字节）编号**表示，那么PQ代码仅为`96字节/向量`。理论代码大小比例`3072/96=32倍`。注意：**这只是PQ编码本身的字节对比**，不计码本、ID、其他索引结构、原文、副本和可能保留的全精度向量；真实系统总存储绝不能直接宣称减少32倍。

7.3 PQ如何影响检索质量?

近似距离会产生“量化误差”。常见思路是先用压缩向量快速筛候选，再取得原始或高精度向量重新排序。若原向量未保留，则无法直接进行完全一致的精确重排。PQ与IVF可以组合成IVF-PQ类索引。**分组检索误差、量化误差、原文/状态错误属于不同来源的失效**。

八、向量索引与向量数据库并不完全是同一个东西

向量索引专门支持高效相似性查找，可视为“根据向量快速找到ID”的数据结构；**向量数据库/向量搜索系统**一般还要处理存储与持久化、元数据、更新删除、过滤、并发、故障恢复、可观测性与查询API等功能。但具体产品的能力存在差异，不能仅凭“vector database”名称就假定它有完整事务保障。

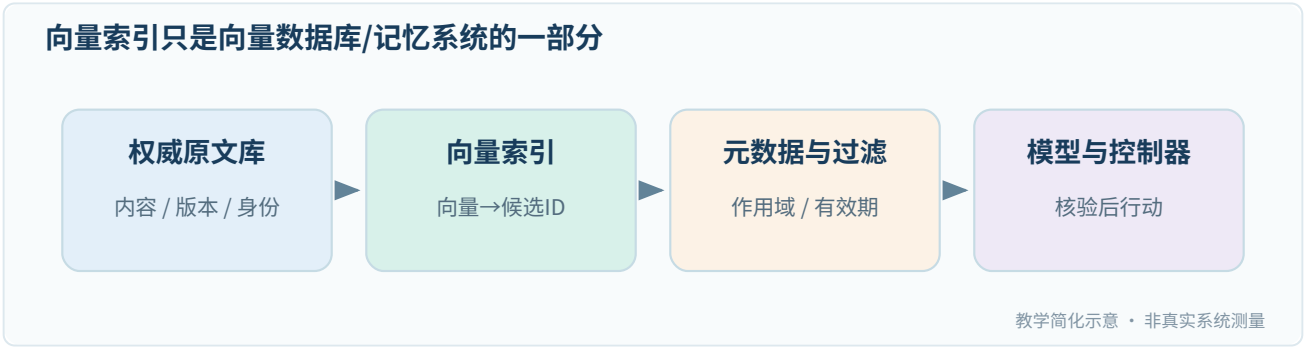


图12 向量索引与长期记忆系统｜教学机制示意，非实际模型运行截图

8.1 一条完整记忆应该有的东西

课堂记忆卡示例：

`memory_id=M018`； `source_event_id=E018`； `content=暂停未经授权的付费实验`； `speaker=项目负责人`； `valid_from=D18`； `status=ACTIVE`； `project=P`； `permission_scope=...`； `embedding_model_version=EMB-v1`； `embedding=[向量]`。

其中几项尤其关键：记忆ID让向量命中能够找到原文；来源ID让主张可以审计；时间与状态使系统识别新旧版本；模型版本说明向量如何生成；项目作用域与权限约束防止跨任务和跨用户数据混淆。

8.2 Faiss、pgvector与完整服务的不同定位

Faiss主要是向量相似搜索与聚类库/索引工具；**pgvector**是在PostgreSQL中增加向量类型、距离计算和索引能力；还有专用向量检索服务/数据库。这是架构选项，而非排名或购买建议。谁适合取决于现有数据、过滤、更新、一致性、规模与运维约束。Faiss并不会替你定义哪些记忆是最新有效政策；pgvector即使嵌入事务数据库，跨外部Embedding服务与缓存仍须另行设计一致性。[1][2][4]

九、检索的实际管线：候选、过滤、重排、核验

一条长期记忆检索路径可以分成：①解析用户问题和当前项目作用域；②把查询编码为向量；③向量索引找Top-N候选；④按合法访问范围、状态与时间过滤；⑤取得原文和来源；⑥必要时重排/融合关键词证据；⑦构建上下文交给模型；⑧控制器再次检查实际执行权限。

重要细节：过滤在索引之前、之中还是之后，会改变质量与延迟。若先全库ANN找10条，结果中只有1条属于当前项目，那么再筛选后只剩1条；并不能因为用户要求Top-5就自动保证有5条符合条件。



图13 过滤时机与候选不足｜教学机制示意，非实际模型运行截图

9.1 过滤的三种直觉

预过滤：先通过权限/项目/有效期确定可搜索的子集，再在子集做向量搜索，有助于隔离但实现与性能依赖索引能力。**后过滤**：先找一批向量近邻，再丢弃不满足元数据的记录，可能导致有效候选不足。**混合/迭代扫描**：扩大搜索、分区或按索引内部能力过滤，以兼顾质量与性能。pgvector官方文档指出特定近似索引的过滤可能在扫描后进行，并提供迭代扫描等机制缓解不足。[4]

9.2 一道简单概率直觉题

假设（**仅为教学独立性近似**）ANN返回的10条候选中，每条满足当前项目/状态条件的概率都是10%。符合条件的期望条数是`10×0.10=1`，**不意味着必然恰好得到1条**，也不保证Top-5可以填满。条件越稀疏，后过滤可能越难满足目标K。

9.3 权限约束不要交给相似度分数

“用户明确暂停付费”属于可能影响执行的授权边界。就算向量搜索第1名是一份“过去如何收费调用API”的教程，控制器也不能把它解释为当前执行批准。**检索层可以影响候选的集合；权限和外部动作必须由有权威状态的逻辑和控制器再次核验。**

十、记忆会变化：增量写入、更新、删除与过时索引

向量记忆不是静态百科全书。用户新的决定、工具的新结果或项目重命名，都可能改变长期记忆。如果正文数据库已经更新为“暂停付费”，但向量索引依然指向旧提议，就会出现**索引新鲜度问题**。



图14 状态和索引更新的时间差｜教学机制示意，非实际模型运行截图

10.1 典型故障并不只有“向量找错了”

- **更新延迟**：权威状态已改变，派生向量索引尚未刷新。
- **旧版本残留**：新旧文本都能被命中，却未建立 SUPERSEDES 替代关系。
- **删除残留**：正文被删，缓存/索引里仍可能留有候选；从索引中不可检索与所有备份永久清除不是同一回事。
- **模型迁移**：Embedding v1与v2的表示空间变了，旧索引混用会影响结果，需要定义重编码、双索引或渐进迁移规则。
- **权限变化**：项目访问权撤销，但旧缓存未同步更新。

解决思路不是一律要求昂贵强一致，而是按风险分层：历史聊天搜索可以接受短暂延迟；付费、删除或隐私权限等关键边界必须在最终执行前读取权威状态。实现时区分单数据库事务与跨系统更新的现实边界。

10.2 “写入成功”的精确定义

“把原始事件保存成功”“更新当前有效状态成功”“向量索引可查询”“缓存刷新完成”“用户端看到最新结果”是不同验收点。只看一个成功标志就宣布整个记忆系统已经更新，是典型工程误判。

十一、速度和成本怎么估算？从空间算例到查询延迟

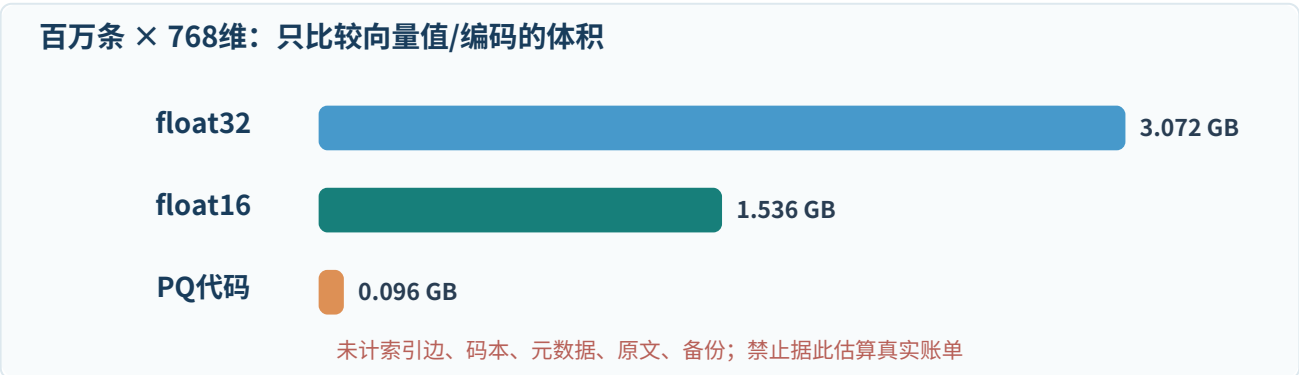


图15 单纯向量编码的空间比较｜教学机制示意，非实际模型运行截图

11.1 原始向量容量

以一百万条、768维为例：float32约**3.072 GB**，float16编码的向量值约**1.536 GB**，PQ96字节编码约**96 MB**。这三个数字仅比较向量值/编码本身，不代表全系统物理文件大小，也不是实际产品账单。HNSW图边、IVF码本、PQ码本、用户文本、冗余副本及数据库索引都会增加体积。

11.2 为什么查询延迟不能只看ANN?

总时延约等于：查询编码 + 元数据/权限处理 + 索引搜索 + 原文回源 + 重排 + LLM阅读推理 + 工具动作。即便ANN从100毫秒降到10毫秒，如果LLM回复要几秒，总任务加速可能非常有限。并且一旦漏掉关键决策，节约的几十毫秒可能导致昂贵错误。

用课堂虚构数据：Embedding 40ms、ANN 15ms、过滤回源20ms、重排50ms、模型推理900ms，总计`40+15+20+50+900=1025ms`。把ANN从15ms优化到5ms，总计1015ms，只改善约`(1025−1015)/1025≈0.98%`。这是理解端到端瓶颈的**纯算术例子**，不代表任何产品的实测速度。

十二、如何选择架构：不预设“越复杂越好”

下面是面向理解设计空间的比较，而不是部署处方。选型时优先回答：N有多大？QPS是多少？写入与删除有多频繁？是否需要严格隔离和元数据过滤？能容忍多少漏检？保存和维护成本是多少？

方案	核心机制	相对优势	主要代价/限制
精确Flat	比较全部候选	真值清晰、实现简单；适合作基线	N与d增长时查询成本上升
HNSW	多层近邻图导航	常有良好召回—延迟权衡	图内存、建索引和更新维护
IVFFlat	聚类分区、查询部分列表	索引相对简洁，可控分区探查	边界近邻遗漏、聚类与训练依赖
PQ/IVF-PQ	子空间量化编码	降低向量码存储与带宽	量化误差、码本训练、可能需精排
磁盘ANN	让部分索引驻留SSD	可扩展超内存数据量	依赖存储IO与具体工作负载

DiskANN论文展示了面向SSD的十亿级ANN设计，说明“向量索引必须全部留在RAM里”并非普遍成立；但论文中的历史硬件和数据集数字不应挪用成当前任何产品的性能承诺。[6]

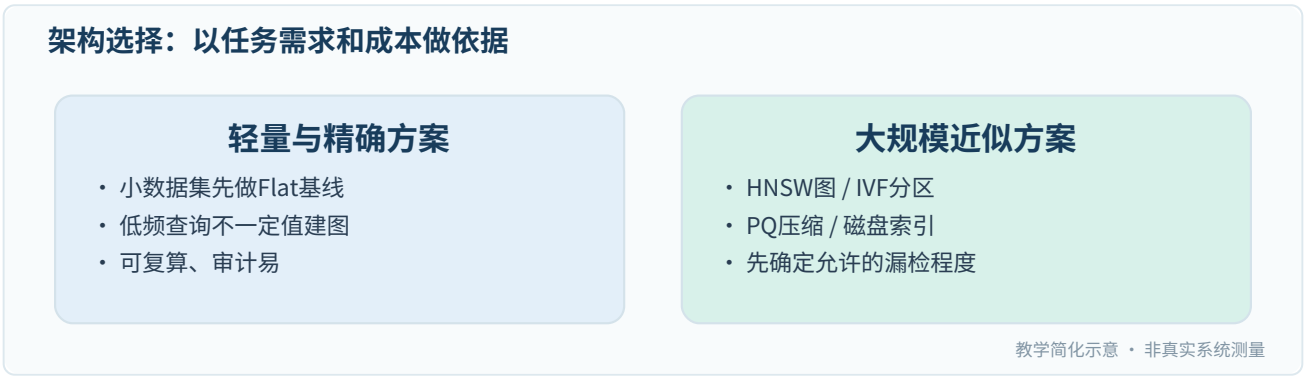


图16 简单基线与复杂索引｜教学机制示意，非实际模型运行截图

十三、怎么评价一个向量记忆系统？不能只看检索分数

至少分三层：

索引层：ANN Recall@K（对比精确最近邻）、查询延迟p50/p95、QPS、索引构建/更新成本、内存和磁盘占用。这层评测算法的加速和遗漏。

记忆层：当前有效记忆的Recall/Precision、时间和版本准确性、来源支持率、过滤/删除传播、用户/项目隔离。这层评测是否找对了**该找的事实**。

任务层：最终回答真实性、拒答适当性、是否遵守有效决定、工具调用和任务成功率、全链路时间/Token/费用。这层评测记忆是否帮助智能体**做对了事**。



图17 三层评价边界｜教学机制示意，非实际模型运行截图

13.1 一个反例说明三层必须分开

系统A的ANN Recall@10=0.98，但高频召回旧提议、忽略最新决定，最终付费决策准确率差。系统B的ANN Recall@10=0.90，但严格检查版本/权限并用混合检索找回有效状态，最终决策可能更可靠。两个数是**教学假设**，不是实证规律；它说明索引分数不能代替下游评测。

13.2 为什么长期记忆基准不只问语义相似度？

LongMemEval（ICLR 2025）将信息提取、多轮会话推理、时间推理、知识更新、拒答等列为核心能力，并把长期记忆设计拆成索引、检索与阅读等阶段。这能提醒我们：记忆系统的目标不是单纯检索几个语义相似句子，而是跨时间维护、恢复和正确使用历史知识。[7]

十四、完整的30天科研智能体动态案例

把前面机制串成真实的“工作流程模型”，注意所有事件均为模拟。

步骤一：写入。第05天AI提出“未来或可扩样使用API”。系统记作`kind=PROPOSAL`、`actor=AI`、`status=UNAPPROVED`，原文持久保存并编码索引；不是项目决定。

步骤二：新决定。第18天项目负责人正式暂停未经重新授权的付费实验。系统写入权威事件M018、更新当前状态，并将对应向量放入索引；旧提议保留用于历史问答，但不能继续作为行动许可。

步骤三：查询。第30天问“现在能不能调用付费API？”检索器把自然语言变成查询向量，ANN可能找出第05天与第18天的记录，并结合项目/时间/状态条件补充词项检索。

步骤四：回源与核验。用`memory_id`取回原文和出处，确认第18天限制仍有效、没有更新的授权决定；记录不确定项。若索引未命中M018，还应该允许按`project+current_policy`从权威状态读取。

步骤五：回答与行动。模型应给出当前限制，说明如需付费必须有新授权；控制器在实际调用工具前再次校验，不因某个检索结果“相似度很高”而自行执行。

步骤六：反馈更新。如果用户后来给出有范围、有预算的新许可，记录时间、边界、版本；同步原始库、有效状态与派生索引，保留审计链。



图18 长期科研智能体端到端案例｜教学机制示意，非实际模型运行截图

14.1 五种容易发生的不同故障

观察到的异常	更可能的环节	不应直接得出的结论
数据库有M018，ANN候选没有	向量索引/召回/编码版本	大模型必然“忘了”
候选有M018，但被过滤掉	项目、状态、时间条件或过滤策略	一定是Embedding错误
选中旧提议而忽略新决定	排序、状态/版本融合	ANN算法本身一定差
回答正确，却自动付费调用	控制器/权限和工具契约	记忆已经足够可靠
新决定已入库，部分会话仍见旧值	缓存、索引同步或会话快照	“主数据库保存失败”

十五、12个零基础常见误区

1. **“向量数据库能理解事实真假。”** 它主要负责相似检索和相关数据服务，真实性需要证据与状态检查。
2. **“向量越接近，结论越一致。”** 两句意义相反的文字也可能在同一主题区域内接近。
3. **“HNSW就是一种Embedding模型。”** HNSW是已存在向量的索引/搜索算法，不负责生成文本Embedding。
4. **“ANN一定比精确搜索更适合所有规模。”** 小规模、低频查询或要求严格精确近邻时，Flat可能更简单。
5. **“调大efSearch就不会有错误。”** 可能减少索引遗漏，却不会纠正来源、权限、时间和语义误判。
6. **“IVF按‘语义类别’人工分组。”** 聚类依据向量几何，不能替代业务分类和授权状态。
7. **“PQ把向量压缩32倍就把整个数据库缩小32倍。”** 编码大小比例不包括码本/图/ID/原文/副本。
8. **“过滤总是在ANN之前进行。”** 实际顺序和机制取决于索引与系统实现。
9. **“删除正文就等于所有副本瞬间不存在。”** 索引、缓存、备份和审计记录有不同更新与保留策略。

10. **“换一个Embedding模型不需要重建索引。”** 模型版本与语义空间变化可能使旧向量不可比较。
11. **“ANN Recall@K高，回答一定正确。”** 高召回只衡量特定近邻集合的恢复，不保证当前有效状态和行动正确。
12. **“买专用向量数据库比用现有数据库更科学。”** 选型应以需求、基线、公平评估和维护成本为依据。

十六、理论思考题（先独立推导，再看答案）

题1：一条记忆向量已经保存在文件里，为什么智能体仍可能“想不起来”？分别给出两个发生在检索层、一个发生在使用层的原因。

题2：100万条768维float32向量，理论原始向量字节数是多少？折合GB与GiB是多少？为什么它不是数据库总容量？

题3：精确Top-5为[A,B,C,D,E]，ANN结果[A,C,D,F,G]，求Recall@5。能否据此说“回答准确率60%”？

题4：q=(1,0)，A=(2,0)，B=(0.8,0.6)，分别比较它们对q的余弦相似度和欧氏距离。结果是否一定相同？

题5：用一个“街区导航”的比喻解释HNSW高层与底层图的区别。再说明这个比喻在哪一点上可能误导初学者。

题6：HNSW中M、efConstruction、efSearch各在哪个阶段主要发挥作用？为什么调大查询参数不能纠正旧记忆被当作新决定？

题7：IVF把12个教学向量分为3组，每组4条。nprobe=1与3对访问范围有什么区别？这能推导真实系统“固定提升3倍”吗？

题8：768维float32向量用96个8bit PQ编码表示，计算原始值与PQ码的字节数及代码压缩比例。实际总系统一定按此比例变小吗？

题9：某近似搜索先返回10条，其中只有1条属于当前项目；但用户要求5条。请说明后过滤会发生什么问题以及两个解决思路。

题10：权威数据库已经保存“暂停付费”决定，但ANN仍召回旧提议。列出至少四个可能的系统因素，并指出哪个环节应在真正付费前核验权限。

题11：一次任务中Embedding 40ms、ANN 15ms、过滤回源20ms、重排50ms、LLM 900ms。若ANN减少10ms，端到端大约优化百分之多少？说明这一数字为何是教学算例。

题12：设计一份“检索层—记忆层—任务层”三级评价清单，判断新HNSW索引是否真的让长期科研助手更可靠；至少给出每层两个指标。

十七、思考题完整答案

答1：保存仅代表可持续存在。检索层可能（a）查询Embedding偏离相应记忆，未进入Top-K；（b）索引延迟或错误的项目过滤排除了记录。使用层可能找到了记忆，但模型忽略它、误读否定词，或者控制器执行了与其不符的动作。定位时先看候选ID再看上下文和工具日志。

答2：$1{,}000{,}000\times768\times4=3{,}072{,}000{,}000$字节。十进制GB为3.072，二进制GiB=$3{,}072{,}000{,}000/1024^3\approx2.861$。不含ID、向量图边、码本、正文、元数据、WAL、缓存或备份；不同数据库的布局还会有额外开销。

答3：交集A、C、D有3条，`Recall@5=3/5=0.6=60%`。这里只评价近似索引是否找回精确Top-5；回答准确度还依赖真实问题、来源状态、阅读与推理，不可直接等同。

答4：`cos(q,A)=2/(1×2)=1`，`cos(q,B)=0.8/(1×1)=0.8`。欧氏距离：`d(q,A)=1`，`d(q,B)=sqrt((1−0.8)²+0.6²)=sqrt(0.4)≈0.632`。余弦A优先、L2　B更近；因此未归一化时不同指标可能给出不同排序。

答5：高层如高速路，通过较少点跨区域；底层如街巷，覆盖更多点做精细寻找。容易误导的一点是以为HNSW图边由现实地理位置建成；实际由向量距离及索引构建算法决定，候选搜索也并非单路径贪心。

答6：M关系到图连边规模，efConstruction主要影响建图候选搜索，efSearch影响查询候选搜索。efSearch加大或可召回更多向量，却不会改变原始主张的身份、有效期或权限状态；后者属于事实/状态核验责任。

答7：nprobe=1仅搜索一个列表，本例4条；nprobe=3搜索全部3组，本例12条。搜索多组可能减少跨分区遗漏，但真实列表大小不均、索引开销和缓存不同，不可能据此推导固定3倍速度或召回变化。

答8：原始`768×4=3072字节`；PQ码`96×1=96字节`；向量码比例`3072/96=32`。码本、ID、图/倒排索引、正文、复制和保留的高精度向量都占额外空间，所以不等于全系统32倍缩小。

答9：先ANN后过滤导致有效候选不足，无法满足Top-5；可以考虑扩大初始候选N/迭代扫描，或让权限和项目范围在检索前/检索内部生效，再于权威状态校验。不能为了凑齐5条而返回不符合权限的记忆。

答10：旧索引未刷新；Embedding版本混用；旧条目未标记`SUPERSEDES`；检索没有加当前状态过滤；缓存滞后；新记忆尚未完成编码；搜索只按语义相似而不查权威状态。真正付费前必须由控制器或对应执行边界核验当前授权，而不能靠ANN排名。

答11：原总耗时`40+15+20+50+900=1025ms`，改后1015ms；优化比例`10/1025≈0.9756%`，约0.98%。这些子阶段时间完全是假设值，不是对特定模型/服务的基准测试。现实应测多次和尾延迟，且任务可能并行。

答12：索引层：ANN Recall@K、p95检索延迟、内存/构建成本；记忆层：有效版本召回率、来源支持率、时间一致率、跨项目隔离率；任务层：最终回答准确率、执行授权违规率、任务成功率、端到端费用/延迟。比较新旧方案需在相同数据、模型、问题集合、过滤和预算口径下进行，不能只凭一个指标宣布改进。

十八、本讲总结：建立可独立重建的整体模型

请闭上教材，用自己的话重画：**原始事件存储→候选记忆编码→向量库保存ID与Embedding→ANN索引加速候选→过滤与回源→版本/来源/权限核验→模型阅读→控制器行动→反馈更新与索引维护**。在箭头上写明每一步的失败点，再把HNSW/IVF/PQ放进正确位置。

对研究者而言，一个最重要的结论是：**高速相似搜索是长期记忆的检索基础设施，而不是长期记忆的全部能力。**HNSW/IVF/PQ解决的是如何高效找到向量候选；如何维护时间、来源、冲突、权限以及最终行动可靠性，是更高层的系统问题。

下一讲（第19讲）：图记忆、知识图谱与时间图谱——智能体怎样保存实体、关系和演化历史？仍为零基础理论课，不安排实验。

参考文献与可核验原始资料

[1] Faiss Team. **Faiss: Efficient similarity search and clustering of dense vectors**. Official documentation. https://faiss.ai/ （索引抽象、Flat精确检索、近似检索、CPU/GPU等）

[2] Johnson, J., Douze, M., & Jégou, H. (2019). **Billion-scale similarity search with GPUs**. IEEE Transactions on Big Data. DOI: 10.1109/TBDATA.2019.2921572. https://ieeexplore.ieee.org/document/8733051 （大规模向量检索、GPU、量化与选择算法）

[3] Malkov, Y. A., & Yashunin, D. A. (2020). **Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs**. IEEE TPAMI; arXiv:1603.09320. https://arxiv.org/abs/1603.09320 （HNSW分层图结构与导航）

[4] **pgvector: Open-source vector similarity search for Postgres**. Official project documentation. https://github.com/pgvector/pgvector （精确与HNSW/IVFFlat检索、过滤与索引参数说明；以访问时版本为准）

[5] Jégou, H., Douze, M., & Schmid, C. (2011). **Product quantization for nearest neighbor search**. IEEE TPAMI 33(1):117–128. DOI:10.1109/TPAMI.2010.57. https://pubmed.ncbi.nlm.nih.gov/21088323/ （PQ原始论文）

[6] Subramanya, S. J., et al. (2019). **DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node**. NeurIPS 2019. https://www.microsoft.com/en-us/research/?p=634449 （SSD驻留图索引的代表研究）

[7] Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.-W., & Yu, D. (2025). **LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory**. ICLR 2025. https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html （长期记忆评估中的索引—检索—阅读与时间/更新）

[8] Faiss Team. **Guidelines to choose an index**. Official wiki. https://github.com/facebookresearch/faiss/wiki/Guidelines-to-choose-an-index （精确索引与规模、内存、查询次数的实务权衡）

[9] Faiss Team. **Faiss Wiki – Introduction and index foundations**. https://github.com/facebookresearch/faiss/wiki （IVF、PQ等方法的历史来源与索引选择）

参考文献使用边界：教程以文献和官方文档解释底层机制；涉及课堂向量、时延、排名、成本和项目日期的具体数字均为自行构造的教学例子。没有进行任何实际性能测试，也没有把某种索引宣布为普遍最优。